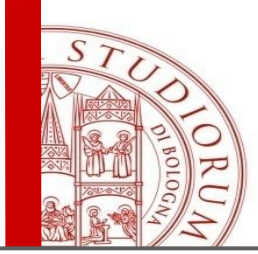
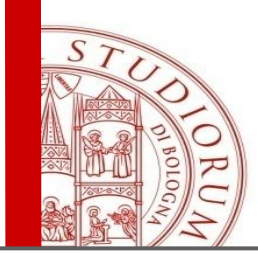


Fondamenti di Computer Graphics (modulo 2)

ACADEMIC YEAR 2025/2026

Intro Three.js

Serena Morigi
Office: F2, 4th floor
serena.morigi@unibo.it

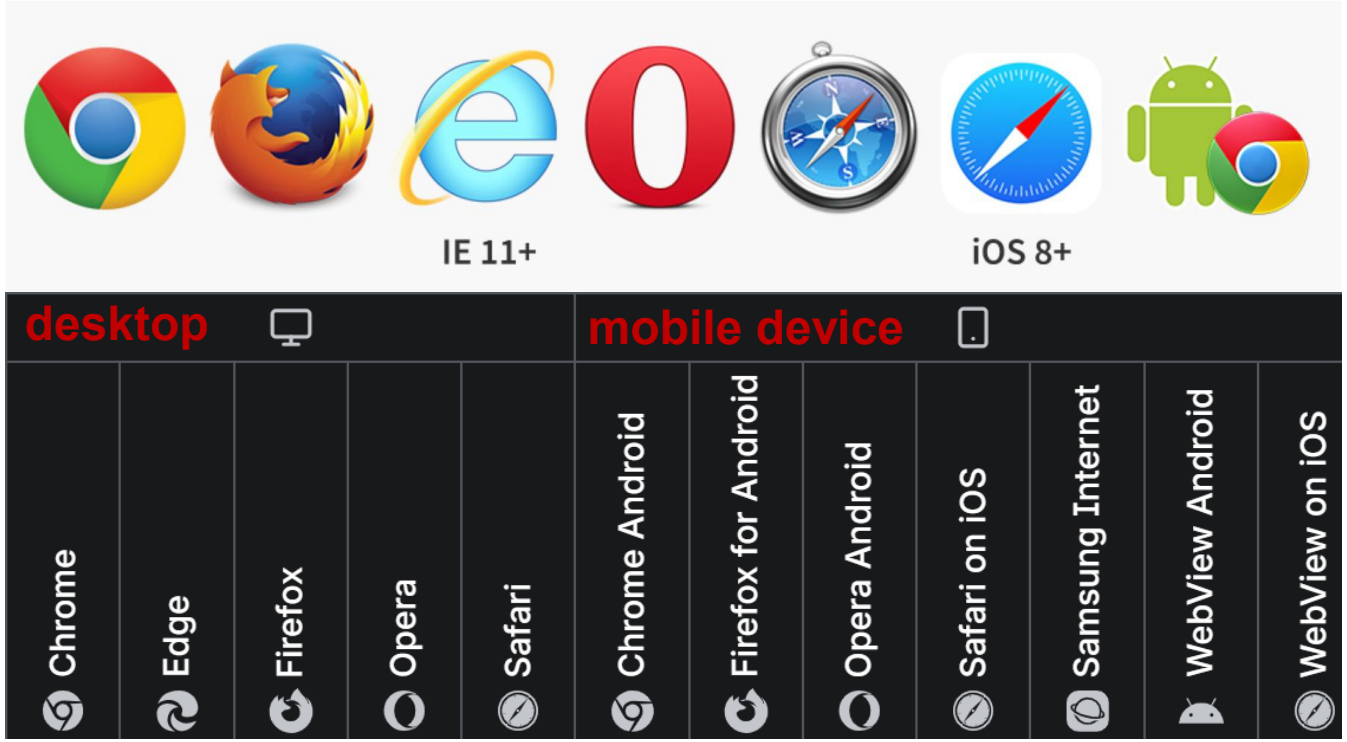


Three.js

- **Three.js** is an open-source, **JavaScript 3D library** that simplifies the process of creating and rendering 3D graphics in the browser using **WebGL** for building interactive 3D scenes.
- It supports **scenes, cameras, lights, materials, geometries, textures, animations, and physics integrations**, for web-based 3D applications, games, and data visualizations.
- Repository: GitHub - mrdoob/three.js: JavaScript 3D Library. · GitHub

WebGL

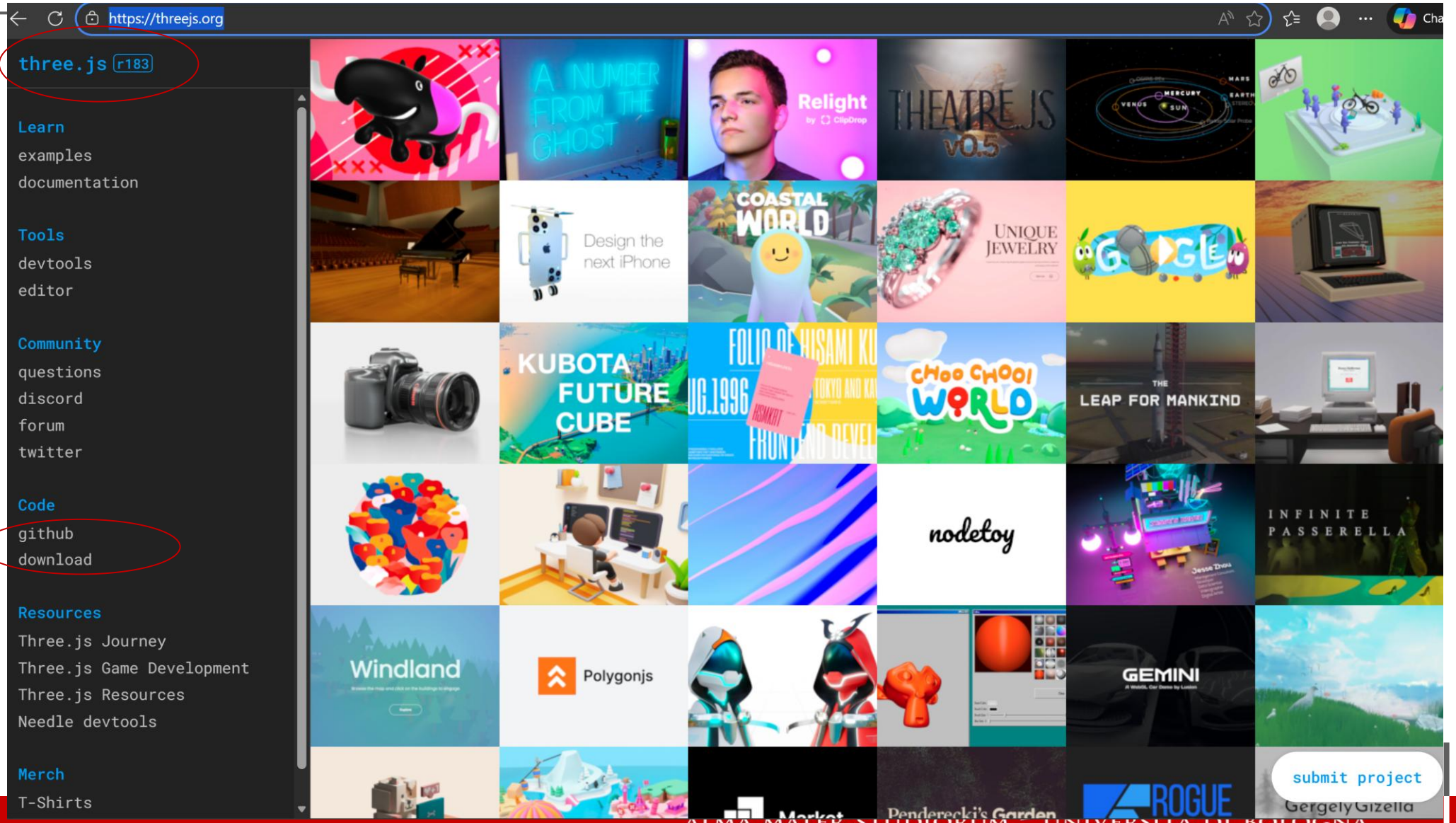
- WebGL: 2D and 3D graphics for the web
- Browser compatibility



Three.js home page

version

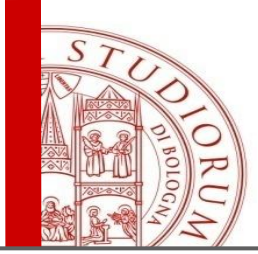
download



Tools you need to work with Three.js

- To create Three.js WebGL applications you need a text editor and one of the supported browsers to render the results:
  - **Visual Studio Code**
  - **Firefox** (Chrome,..)
- **Download Three.js**

  *See **running-threejs-examples.pdf***
- **Get the server up and running**
  - `$ npm run serve`



Project structure

- HTML file to define the webpage
- JavaScript file to run your three.js code

index.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>My first three.js app</title>
    <style>
      body { margin: 0; }
    </style>
  </head>
  <body>
    <script type="module" src="/main.js"></script>
  </body>
</html>
```

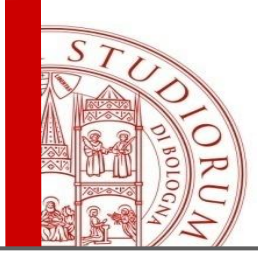
main.js

```
import * as THREE from 'three';

// JavaScript code that uses three.js

...
```

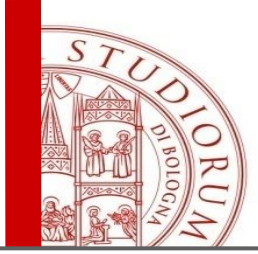
import *everything* from the three.js core into main.js under the THREE namespace



JavaScript Modules

- ..Organize your code in js modules..
- From Revision 106 (2019), three.js uses **modules**
- Modules have the advantage that they can easily import other modules they need. That saves us from having to manually load extra scripts they are dependent on.

```
<script type="module" >
   import * as THREE from
   'https://cdn.jsdelivr.net/npm/three@0.178.0/build/three.module.js';
</script>
```

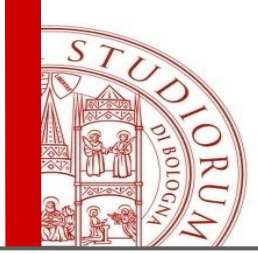


JavaScript Modules

As JavaScript applications began to grow in size, the need to organize code into **modules** became apparent. A module is simply a JavaScript file.

Modules can load one another and use special **export** and **import** directives to exchange functionality, allowing functions to be called from one module to another:

- **export** marks variables and functions that should be accessible from outside the current module.
- **import** allows for the importing of functionality from other modules.



JavaScript Modules

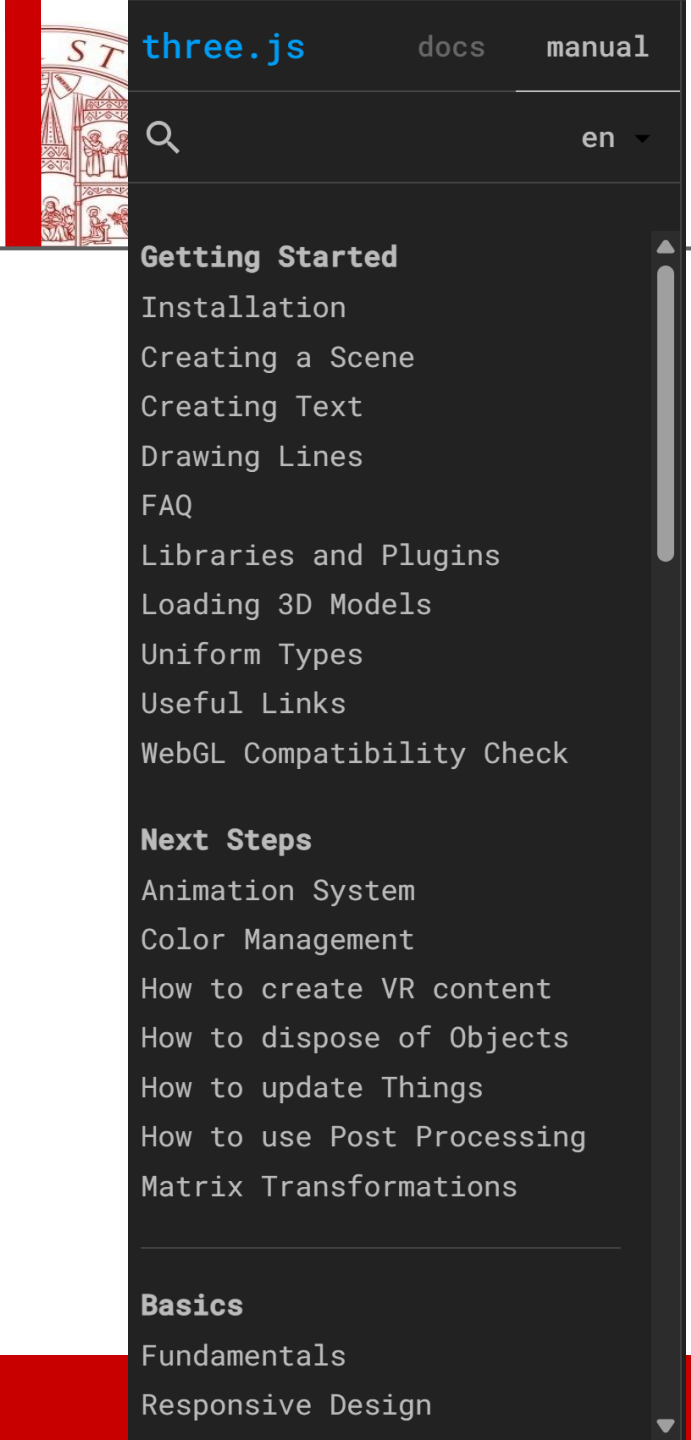
- For example, if we have a file named **`diCiao.js`**, we can make the function available externally (export it) like this:

```
// file diCiao.js
export function diCiao(user) {
  alert(`Hello, ${user}!`);
}
```

- ...subsequently, another file can import and use it like this:

```
// file main.js
import { diCiao } from './diCiao.js';
diCiao('John'); // Hello, John!
```

- The import directive loads the module located in the ./diCiao.js file into the current file and assigns the exported `diCiao` function to the corresponding diCiao variable.

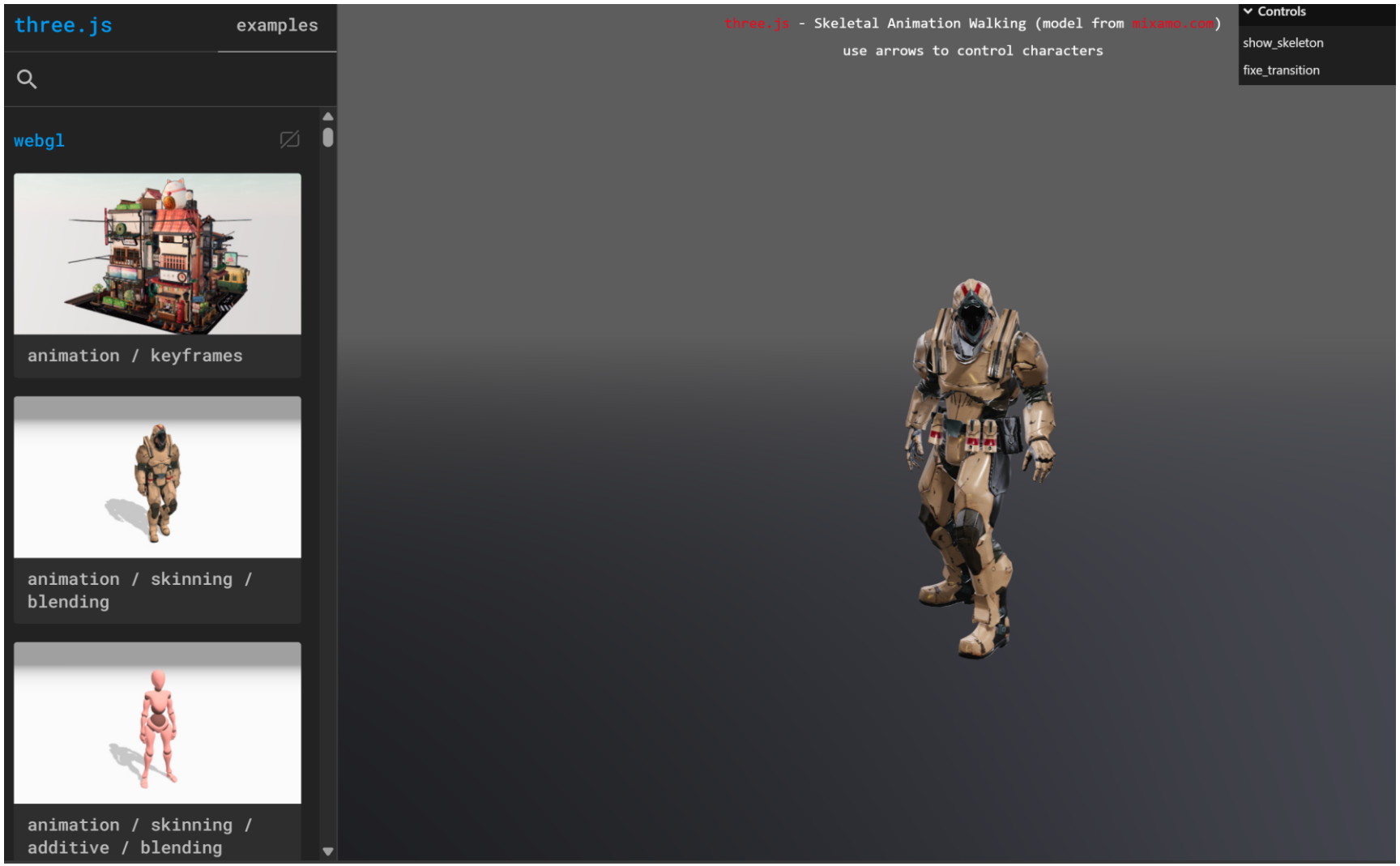


Documentation

- three.js manual

- Let's go to the **three.js** home page and click on 'Documentation.'
- As you can see, the online **three.js Manual** opens, followed by a precise and detailed list of references

Examples



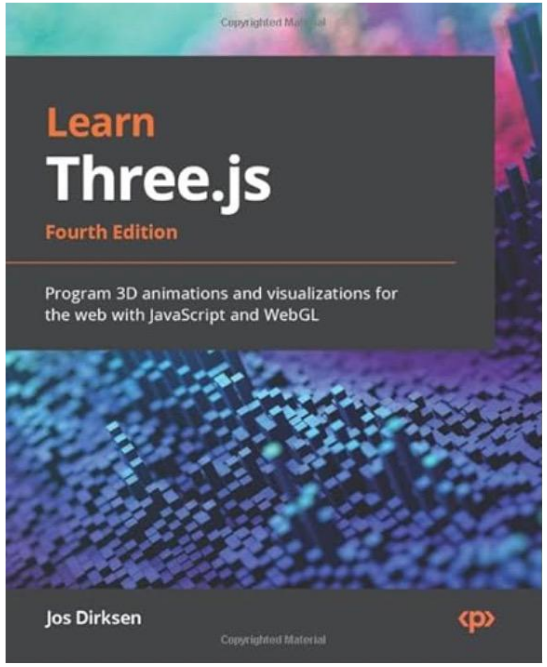
- Let's head back to the **three.js** home page and click on 'Examples.' This opens an extensive showcase of online examples that clearly demonstrate the library's potential.

- The final six hours of this module will be dedicated to analyzing some of these examples, but more importantly, the **source code** behind them.

Resources

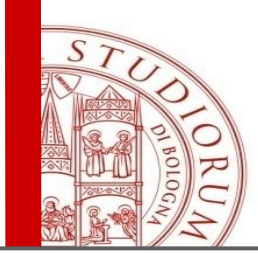
```
Resources
Three.js Journey
Three.js Game Development
Three.js Resources
Needle devtools
```

Returning to the **three.js** home page, we can see the **Resources** menu:
Three.js Game Development: an online advanced course for gaming;
Three.js Journey: an online course consisting of chapters, lessons, and videos;



Learn Three.js: a physical book available for purchase, which includes a code archive to download and use (we will be adopting this one)

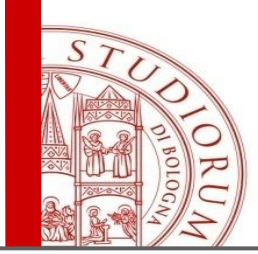
Textbook : “Learning Three.js”, Jos Dirksen, fourth edition
You can download the example code files for this book from GitHub at
https://github.com/PacktPublishing/Learn-Three.js-Fourth-edition



A dive into the code

Returning to the **Three.js distribution** (see the download section), let's take a look at what the existing folders contain:

- **build**: contains the 3D engine, specifically the three.module.js and three.module.min.js files.
- **docs**: contains an index.html file that, when run on a local web server, displays the **Documentation** from the Three.js home page.
- **editors**: contains an index.html file that allows you to use the **Editor** application found on the Three.js home page.
- **examples**: contains an index.html file that presents the **Examples** from the Three.js home page.
- **manual**: contains an index.html file that displays the **Manual** from the Three.js home page.
- **playground**: contains an index.html file that hosts an interesting application.
- **src, test, utils**: contain useful files that we could describe as **external plugins**.



Add-ons

- **three.js** includes the core fundamentals of a 3D engine. Other components, such as controls, loaders, and post-processing effects, are part of the addons/ directory (see examples/jsm/Addons.js).
- These add-ons do not need to be installed separately since they are included in the distribution; however, they must be **imported separately**.

- The following example demonstrates how to import **three.js** along with the **GLTFLoader** add-on.
- the **GLTFLoader** is the standard and recommended way to import 3D models. GLTF (GL Transmission Format)
- you must import the loader from the examples directory (since it is not part of the core **three.module.js**) and then instantiate it.

GLTFLoader

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

const loader = new GLTFLoader();

loader.load(
    'path/to/model.gltf',
    function (gltf) {
      // Add the loaded scene to your three.js scene
      scene.add(gltf.scene);
    },
    function (xhr) {
      console.log((xhr.loaded / xhr.total * 100)+'% loaded');
    },
    function (error) {
        console.error('An error happened', error);
    }
);
```

Components of the GLTF Object

When the model loads, the gltf object returned in the callback contains:

- **gltf.scene**: The actual Group/Object3D you add to your scene.
- **gltf.animations**: An array of AnimationClip objects for moving parts.
- **gltf.cameras**: Any cameras exported within the 3D file.
- **gltf.asset**: Metadata about the file.

GLTF is optimized for the web. It supports materials, textures, and animations much better than the OBJ format.

Export your 3D object as GLTF from Blender (Recommended)

Instead of using the .obj format, you should export your model directly as a .gltf or .glb from Blender

ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA

Serena Morigi
Department of Mathematics
serena.morigi@unibo.it
https://www.unibo.it/sitoweb/serena.morigi